

FerreSync — Prompt Maestro de Arquitectura y Desarrollo

Proyecto Integrador · Cristian David Cárdenas García · UTS · 2026

CONTEXTO DEL PROYECTO

FerreSync es un sistema de información web para la gestión centralizada de inventario, facturación (POS) y cuentas por cobrar de una ferretería tradicional. El sistema reemplaza el uso de cuadernos físicos y memoria humana por un entorno digital estructurado, con roles de acceso, asistente de IA y arquitectura desacoplada.

ARQUITECTURA GENERAL DEL SISTEMA

Patrón Arquitectónico

```
[Cliente Web (Frontend)] <--> [API REST (Backend)] <--> [Base de Datos (PostgreSQL)]  
[API REST (Backend)] <--> [Storage (Archivos)]  
[API REST (Backend)] <--> [IA / RAG (Chatbot)]
```

Separación de Capas (Bajo Acoplamiento)

- Frontend:** Aplicación web SPA (Single Page Application) — consume la API via HTTPS/JSON
- Backend:** API RESTful con lógica de negocio, autenticación JWT y validaciones
- Base de Datos:** PostgreSQL relacional con integridad referencial
- Storage:** Almacenamiento de archivos (facturas, comprobantes PDF/JPG/PNG)
- IA:** Módulo RAG para el chatbot operativo

STACK TECNOLÓGICO

DESARROLLO LOCAL (Fase inicial — iniciar YA)

Capa	Tecnología Local	Razón
Frontend	React + Vite + TailwindCSS	SPA rápida, componentes reutilizables
Backend	Node.js + Express.js	API REST ligera y flexible

Capa	Tecnología Local	Razón
Base de datos	PostgreSQL (local via Docker o instalación directa)	Mismo motor que Supabase en producción
ORM	Prisma ORM	Migrations, type-safety, fácil swap a Supabase
Autenticación	JWT (jsonwebtoken) + bcrypt	Tokens seguros, encriptación de contraseñas
Storage local	Multer + carpeta <code>/uploads</code> local	Mismo contrato de API, swap fácil a Supabase Storage
IA/Chatbot	LangChain.js + Ollama (local) o OpenAI API	RAG sobre documentos del sistema

PRODUCCIÓN FUTURA (Adaptación rápida — mismo código base)

Capa	Tecnología Producción	Cambio requerido
Base de datos	Supabase (PostgreSQL hosted)	Cambiar <code>DATABASE_URL</code> en <code>.env</code>
Storage	Supabase Storage	Cambiar servicio de upload (1 archivo)
Autenticación	Supabase Auth (opcional) o mantener JWT propio	Opcional, JWT propio es suficiente
Deploy Backend	VPS Contabo (Node.js + PM2 + Nginx)	Docker Compose o script de deploy
Deploy Frontend	Vercel / Netlify / VPS mismo	<code>npm run build</code> → deploy estático
IA	OpenAI API + embeddings en Supabase pgvector	Cambiar proveedor LLM en config

📁 ESTRUCTURA DE DIRECTORIOS DEL PROYECTO

```
ferresync/
├── frontend/           # React + Vite + TailwindCSS
│   ├── src/
│   │   ├── components/ # Componentes reutilizables UI
│   │   └── Layout/      # Sidebar, Topbar, Layout general
```

- ├── POS/ # Componentes del punto de venta
- ├── Inventario/ # Catálogo, formularios de stock
- ├── Cartera/ # Clientes, estados de cuenta, abonos
- ├── Cronograma/ # Calendario interactivo
- ├── Configuracion/ # Personal, reportes, ajustes
- ├── Chatbot/ # Widget de IA flotante
- ├──  pages/ # Páginas/vistas principales
- ├── Login.jsx
- ├── Dashboard.jsx
- ├── PuntoDeVenta.jsx
- ├── Inventario.jsx
- ├── CuentasPorCobrar.jsx
- ├── Cronograma.jsx
- ├── Configuracion.jsx
- ├──  hooks/ # Custom hooks (useAuth, usePOS, etc.)
- ├──  services/ # Funciones fetch a la API (axios)
- ├── api.js # Cliente axios con interceptores JWT
- ├── authService.js
- ├── ventasService.js
- ├── inventarioService.js
- ├── carteraService.js
- ├── iaService.js
- ├──  context/ # AuthContext, AppContext
- ├──  utils/ # Helpers (formatCurrency, formatDate)
- ├──  assets/ # Logos, íconos
- ├── index.html
- ├── vite.config.js
- ├── tailwind.config.js
- ├──  backend/ # Node.js + Express
- ├──  src/
- ├──  routes/ # Definición de rutas Express
- ├── auth.routes.js
- ├── ventas.routes.js
- ├── inventario.routes.js
- ├── cartera.routes.js
- ├── logistica.routes.js
- ├── usuarios.routes.js
- ├── ia.routes.js
- ├──  controllers/ # Lógica de cada endpoint
- ├──  middlewares/ # auth.middleware.js, roles.middleware.js
- ├──  services/ # Lógica de negocio pura
- ├──  prisma/ # Schema y migrations
- ├── schema.prisma
- ├──  uploads/ # Archivos locales (swap a Supabase en prod)
- ├── app.js # Entry point Express
- ├── .env # Variables de entorno

```
| └─ package.json
|
└─ docker-compose.yml          # PostgreSQL local en Docker
└─ README.md
```

MODELO DE BASE DE DATOS (Schema Prisma)

prisma

```
// backend/src/prisma/schema.prisma

generator client {
  provider = "prisma-client-js"
}

datasource db {
  provider = "postgresql"
  url      = env("DATABASE_URL")
}

// ——— USUARIOS Y ROLES ———
model Usuario {
  id          Int          @id @default(autoincrement())
  nombre      String
  correo      String       @unique
  password_hash String
  rol         Rol          @default(CAJERO)
  activo      Boolean      @default(true)
  creado_en   DateTime     @default(now())
  ventas      Venta[]
  cierres_caja CierreCaja[]
}

enum Rol {
  ADMINISTRADOR
  CAJERO
}

// ——— INVENTARIO ———
model Producto {
  id          Int          @id @default(autoincrement())
  sku         String       @unique
  nombre      String
  categoria   String?
  precio_compra Decimal    @db.Decimal(10,2)
  precio_venta Decimal    @db.Decimal(10,2)
  stock_actual Int         @default(0)
  stock_minimo Int         @default(5)
  proveedor   String?
  ubicacion   String?
  url_factura String?      // Ruta archivo o URL Supabase Storage
  activo      Boolean      @default(true)
  creado_en   DateTime     @default(now())
  items_venta ItemVenta[]
  items_devolucion ItemDevolucion[]
}
```

```

}

// — VENTAS (POS) —
model Venta {
    id                Int                @id @default(autoincrement())
    ticket_id         String             @unique // TKT-YYYYMMDD-XXX
    usuario_id        Int
    cliente_id        Int?
    subtotal          Decimal            @db.Decimal(10,2)
    impuestos         Decimal            @db.Decimal(10,2)
    total             Decimal            @db.Decimal(10,2)
    metodo_pago       MetodoPago
    es_credito        Boolean            @default(false)
    estado            EstadoVenta @default(COMPLETADA)
    fecha             DateTime           @default(now())
    usuario           Usuario            @relation(fields: [usuario_id], references: [id])
    cliente           Cliente?          @relation(fields: [cliente_id], references: [id])
    items             ItemVenta[]
    devoluciones      Devolucion[]
}

model ItemVenta {
    id                Int                @id @default(autoincrement())
    venta_id          Int
    producto_id       Int
    cantidad          Int
    precio_unit       Decimal            @db.Decimal(10,2)
    subtotal          Decimal            @db.Decimal(10,2)
    venta             Venta              @relation(fields: [venta_id], references: [id])
    producto          Producto @relation(fields: [producto_id], references: [id])
}

enum MetodoPago {
    EFECTIVO
    TRANSFERENCIA
    CREDITO
}

enum EstadoVenta {
    COMPLETADA
    ANULADA
    DEVUELTA_PARCIAL
}

// — DEVOLUCIONES —
model Devolucion {
    id                Int                @id @default(autoincrement())

```

```

    venta_id      Int
    motivo        String?
    total          Decimal    @db.Decimal(10,2)
    fecha          DateTime    @default(now())
    venta          Venta       @relation(fields: [venta_id], references: [id])
    items          ItemDevolucion[]
}

model ItemDevolucion {
  id              Int          @id @default(autoincrement())
  devolucion_id   Int
  producto_id     Int
  cantidad        Int
  devolucion       Devolucion @relation(fields: [devolucion_id], references: [id])
  producto         Producto   @relation(fields: [producto_id], references: [id])
}

// ——— CLIENTES Y CARTERA ———
model Cliente {
  id              Int          @id @default(autoincrement())
  cedula_nit      String       @unique
  nombre          String
  telefono        String?
  limite_credito   Decimal      @db.Decimal(10,2) @default(0)
  creado_en       DateTime     @default(now())
  ventas          Venta[]
  abonos          Abono[]
}

model Abono {
  id              Int          @id @default(autoincrement())
  cliente_id      Int
  monto           Decimal      @db.Decimal(10,2)
  metodo_pago     MetodoPago
  url_recibo      String?      // Ruta archivo o URL Supabase Storage
  fecha           DateTime     @default(now())
  cliente         Cliente      @relation(fields: [cliente_id], references: [id])
}

// ——— CIERRE DE CAJA ———
model CierreCaja {
  id              Int          @id @default(autoincrement())
  usuario_id      Int
  total_teorico    Decimal      @db.Decimal(10,2)
  total_fisico     Decimal      @db.Decimal(10,2)
  diferencia       Decimal      @db.Decimal(10,2)
  resultado        ResultadoCierre
}

```

```

url_reporte_pdf String?
fecha            DateTime @default(now())
usuario         Usuario   @relation(fields: [usuario_id], references: [id])
}

enum ResultadoCierre {
    CUADRE
    FALTANTE
    SOBRANTE
}

// — LOGÍSTICA / CRONOGRAMA —
model EntregaLogistica {
    id            Int          @id @default(autoincrement())
    titulo        String
    descripcion   String?
    tipo          TipoEvento
    estado        EstadoEntrega @default(PENDIENTE)
    fecha_evento  DateTime
    creado_en     DateTime     @default(now())
}

enum TipoEvento {
    ENTREGA_CLIENTE
    RECEPCION_PROVEEDOR
    RECORDATORIO
}

enum EstadoEntrega {
    PENDIENTE
    EN_RUTA
    ENTREGADO
}

```

SEGURIDAD Y AUTENTICACIÓN

Flujo de Autenticación JWT

1. POST /api/v1/auth/login
 - Valida correo + contraseña (bcrypt.compare)
 - Genera JWT con payload: { userId, rol, nombre }
 - Token expira en 12 horas
 - Respuesta: { token, usuario: { nombre, rol } }

2. Cada petición protegida incluye:
Authorization: Bearer <token>
3. Middleware auth.middleware.js:
→ Verifica y decodifica el JWT
→ Adjunta req.usuario = { id, rol }
→ Si token inválido/expirado → 401 Unauthorized
4. Middleware roles.middleware.js:
→ requiereRol('ADMINISTRADOR') → bloquea CAJERO con 403 Forbidden

Reglas de Seguridad por Rol

Operación	CAJERO	ADMINISTRADOR
Registrar venta	✓	✓
Ver historial propio	✓	✓
Ver historial general	✗	✓
Agregar producto al stock	✗	✓
Eliminar/editar producto	✗	✓
Gestionar personal	✗	✓
Ver reportes de ganancias	✗	✓
Cierre de caja	✓	✓
Procesar devolución	✓	✓
Registrar abono	✓	✓
Ver cronograma	✓	✓
Crear/editar cronograma	✗	✓
Crear clientes	✓	✓
Chatbot IA	✓	✓

Medidas de Seguridad Adicionales

- Bloqueo de cuenta tras 5 intentos fallidos de login por 15 minutos
- Contraseñas hasheadas con bcrypt (salt rounds: 10)
- CORS configurado para solo permitir el origen del frontend

- Validación de inputs con Zod o express-validator en todos los endpoints
 - Rate limiting en rutas de autenticación (express-rate-limit)
 - Archivos subidos: validación de tipo MIME y tamaño máx. 5MB
-

FLUJOS DE TRABAJO PRINCIPALES

Flujo 1: Registrar Venta (POS)

Cajero → Abre "Punto de Venta"

- Busca producto por nombre o SKU (GET /api/v1/productos?search=...)
- Agrega al carrito (estado local React)
- Carrito actualiza subtotal + impuestos en tiempo real
- Selecciona método de pago
- Si CREDITO → selecciona cliente registrado
- Confirma → POST /api/v1/ventas
 - Backend: crea Venta + Items + descuento stock (transacción atómica)
 - Si crédito: verifica límite disponible del cliente
 - Retorna ticket_id y datos para imprimir
- Frontend muestra ticket, opción de imprimir

Flujo 2: Registrar Abono con Comprobante

Cajero/Admin → Selecciona cliente en "Cuentas por Cobrar"

- Ve estado de cuenta (GET /api/v1/cartera/estado-cuenta/:clienteId)
- Hace clic en "Registrar Abono"
- Ingresas monto, método, fecha, adjunta archivo
- Frontend: multipart/form-data → POST /api/v1/cartera/abonos
 - Backend: valida que monto ≤ saldo pendiente
 - Guarda archivo en /uploads (local) o Supabase Storage (prod)
 - Registra abono en BD, retorna nuevo saldo
- Vista se actualiza mostrando saldo reducido

Flujo 3: Cierre de Caja

Cajero → Selecciona "Cerrar Caja"

- Backend calcula total teórico (base + ventas efectivo - egresos)
- Cajero ingresa conteo físico
- POST /api/v1/caja/cierre
 - Calcula diferencia
 - Bloquea nuevas ventas con fecha del turno cerrado
 - Genera PDF del reporte de cierre
- Descarga o imprime reporte

Flujo 4: Asistente IA (Chatbot RAG)

Usuario → Abre widget chat (esquina inferior)
→ Escribe pregunta operativa
→ POST /api/v1/ia/consultar { pregunta: "..."}
→ Backend: genera embedding de la pregunta
→ Busca chunks relevantes en base de conocimiento (manuales del sistema)
→ Construye prompt con contexto + pregunta
→ Llama LLM (Ollama local / OpenAI prod)
→ Retorna respuesta basada exclusivamente en los manuales
→ Respuesta aparece en el chat

🎨 DISEÑO DE INTERFAZ (UI/UX)

Paleta de Colores (basada en mockups)

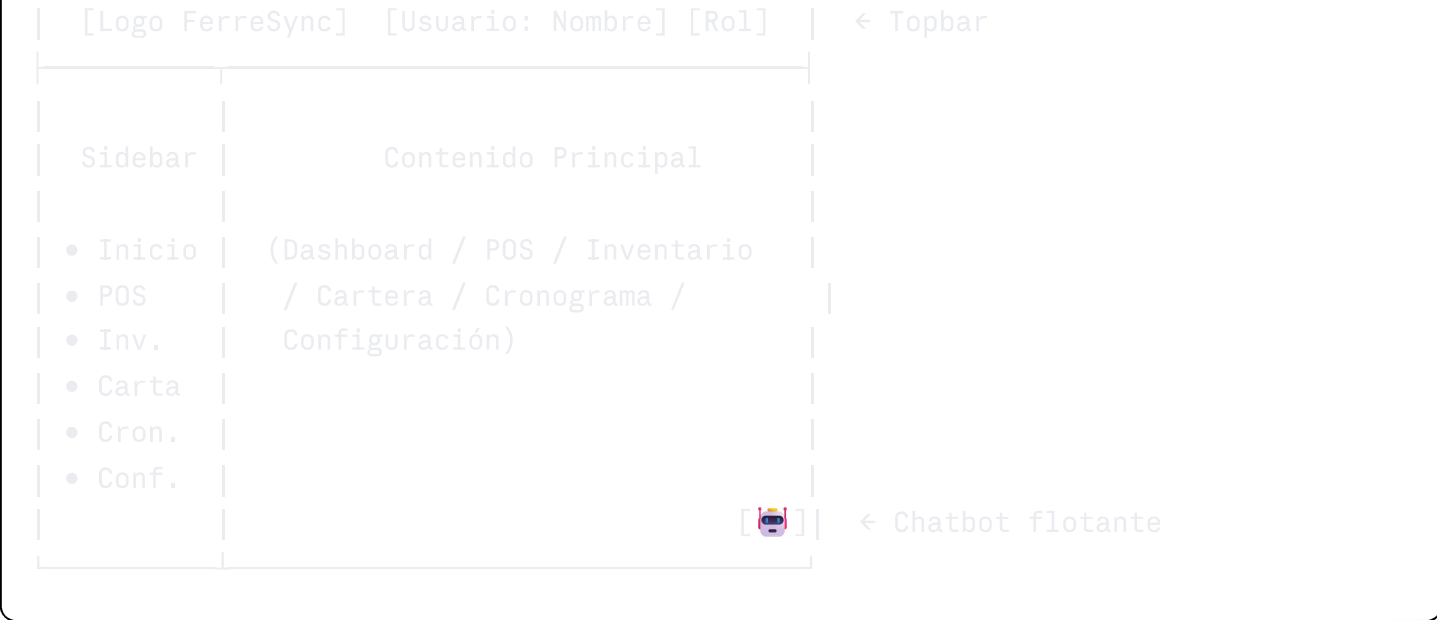
```
CSS

:root {
  --color-primary: #1E3A5F;      /* Azul oscuro ferretería */
  --color-secondary: #F5A623;    /* Amarillo/naranja acento */
  --color-success: #27AE60;      /* Verde stock OK */
  --color-warning: #F39C12;      /* Naranja stock bajo */
  --color-danger: #E74C3C;       /* Rojo stock crítico / mora */
  --color-bg: #0F1923;          /* Fondo oscuro dashboard */
  --color-surface: #1A2733;      /* Cards y paneles */
  --color-text: #ECF0F1;         /* Texto principal */
  --color-text-muted: #95A5A6;   /* Texto secundario */
}
```

Reglas de UX (según IEEE 830 del proyecto)

- Flujos de máximo **3 clics** para registrar una venta
- Botones grandes y legibles (min 44px altura)
- Totales visibles en tiempo real sin necesidad de scroll
- Indicadores de color para stock: Verde ≥ mínimo, Naranja cerca del mínimo, Rojo por debajo
- Saldos en mora (>30 días) resaltados en rojo automáticamente
- Responsive: funciona en PC, tablet y móvil

Layout Principal



📡 ENDPOINTS DE LA API REST

Auth

POST	/api/v1/auth/login	→ Iniciar sesión → JWT
POST	/api/v1/auth/logout	→ Invalidar sesión (token blacklist)

Productos / Inventario

GET	/api/v1/productos	→ Listar (paginado, filtros: search, categoria, ubicacion)
GET	/api/v1/productos/:id	→ Detalle de producto
POST	/api/v1/productos	→ Crear producto [ADMIN]
PUT	/api/v1/productos/:id	→ Editar producto [ADMIN]
DELETE	/api/v1/productos/:id	→ Desactivar producto [ADMIN]
GET	/api/v1/productos/stock-critico	→ Productos bajo mínimo [ADMIN]

Ventas (POS)

GET	/api/v1/ventas	→ Historial (filtros: fecha, cliente, producto) [ADMIN]
GET	/api/v1/ventas/mis-ventas	→ Ventas del cajero actual
POST	/api/v1/ventas	→ Registrar nueva venta
GET	/api/v1/ventas/:id	→ Detalle de venta / ticket
POST	/api/v1/ventas/:id/anular	→ Anular venta [ADMIN]

Devoluciones

POST	/api/v1/devoluciones	→ Procesar devolución
------	----------------------	-----------------------

GET /api/v1/devoluciones/:ventaId → Ver devoluciones de una venta

Cierre de Caja

GET /api/v1/caja/estado-actual → Total teórico del turno actual
POST /api/v1/caja/cierre → Ejecutar cierre y generar PDF
GET /api/v1/caja/historial → Histórico de cierres [ADMIN]

Clientes y Cartera

GET /api/v1/clientes → Listar clientes
POST /api/v1/clientes → Crear cliente
GET /api/v1/clientes/:id → Detalle de cliente
PUT /api/v1/clientes/:id → Editar cliente [ADMIN]
GET /api/v1/cartera/estado-cuenta/:clienteId → Estado de cuenta
POST /api/v1/cartera/abonos → Registrar abono (multipart/form-data)

Logística / Cronograma

GET /api/v1/logistica/cronograma → Eventos del mes (?mes=03&anio=2026)
POST /api/v1/logistica/entregas → Nueva entrega/pedido [ADMIN]
PUT /api/v1/logistica/entregas/:id → Actualizar estado [ADMIN]
DELETE /api/v1/logistica/entregas/:id → Eliminar evento [ADMIN]

Usuarios / Personal

GET /api/v1/usuarios → Listar empleados [ADMIN]
POST /api/v1/usuarios → Crear usuario [ADMIN]
PUT /api/v1/usuarios/:id → Editar usuario [ADMIN]
DELETE /api/v1/usuarios/:id → Desactivar usuario [ADMIN]

IA / Chatbot

POST /api/v1/ia/consultar → Enviar pregunta al chatbot RAG

Reportes

GET /api/v1/reportes/ventas → Reporte ventas (PDF/JSON) [ADMIN]
GET /api/v1/reportes/inventario → Reporte inventario [ADMIN]
GET /api/v1/reportes/clientes → Reporte cartera [ADMIN]

🌸 VARIABLES DE ENTORNO (.env del backend)

```
env

# ——— DESARROLLO LOCAL ———
NODE_ENV=development
PORT=3001

# Base de datos local (Docker PostgreSQL)
DATABASE_URL="postgresql://ferresync_user:ferresync_pass@localhost:5432/ferresync_db"

# JWT
JWT_SECRET=tu_clave_secreta_muy_larga_y_aleatoria_aqui
JWT_EXPIRES_IN=12h

# Archivos locales
UPLOAD_DIR=./src/uploads
MAX_FILE_SIZE=5242880 # 5MB en bytes

# IA Local (Ollama)
IA_PROVIDER=ollama
OLLAMA_BASE_URL=http://localhost:11434
OLLAMA_MODEL=llama3

# ——— PRODUCCIÓN (descomentar al migrar) ———
# DATABASE_URL="postgresql://user:pass@db.supabase.co:5432/postgres"
# SUPABASE_URL=https://tu-proyecto.supabase.co
# SUPABASE_SERVICE_KEY=tu_service_role_key
# SUPABASE_BUCKET=ferresync-archivos
# IA_PROVIDER=openai
# OPENAI_API_KEY=sk-...
```

🚀 INSTRUCCIONES DE INSTALACIÓN LOCAL

Prerrequisitos

```
bash

# Instalar:
- Node.js v20+
- Docker Desktop (para PostgreSQL local)
- Git
```

Paso a Paso

bash

1. Clonar / crear proyecto

```
mkdir ferresync && cd ferresync
```

2. Levantar PostgreSQL con Docker

```
docker run --name ferresync-db \  
  -e POSTGRES_USER=ferresync_user \  
  -e POSTGRES_PASSWORD=ferresync_pass \  
  -e POSTGRES_DB=ferresync_db \  
  -p 5432:5432 \  
  -d postgres:15
```

3. Configurar Backend

```
cd backend
```

```
npm install
```

```
cp .env.example .env
```

→ Editar .env con los valores locales

4. Ejecutar migrations de Prisma

```
npx prisma migrate dev --name init
```

```
npx prisma generate
```

5. Seed inicial (crear admin por defecto)

```
node src/seed.js
```

Crea: admin@ferresync.com / Admin123!

6. Iniciar Backend

```
npm run dev
```

→ API corriendo en http://localhost:3001

7. Configurar Frontend

```
cd ../frontend
```

```
npm install
```

→ Verificar que VITE_API_URL=http://localhost:3001/api/v1 en .env

8. Iniciar Frontend

```
npm run dev
```

→ App corriendo en http://localhost:5173

DEPENDENCIAS CLAVE

Backend (package.json)

json

```
{
  "dependencies": {
    "express": "^4.18",
    "@prisma/client": "^5.0",
    "bcryptjs": "^2.4",
    "jsonwebtoken": "^9.0",
    "multer": "^1.4",
    "cors": "^2.8",
    "dotenv": "^16.0",
    "express-rate-limit": "^7.0",
    "zod": "^3.22",
    "puppeteer": "^21.0",
    "langchain": "^0.1",
    "@langchain/community": "^0.0"
  },
  "devDependencies": {
    "prisma": "^5.0",
    "nodemon": "^3.0"
  }
}
```

Frontend (package.json)

json


```
{
  "dependencies": {
    "react": "^18",
    "react-dom": "^18",
    "react-router-dom": "^6",
    "axios": "^1.6",
    "@tanstack/react-query": "^5.0",
    "react-hook-form": "^7.0",
    "zod": "^3.22",
    "@hookform/resolvers": "^3.0",
    "react-big-calendar": "^1.8",
    "date-fns": "^3.0",
    "recharts": "^2.0",
    "react-hot-toast": "^2.4",
    "lucide-react": "^0.383"
  },
  "devDependencies": {
    "vite": "^5.0",
    "@vitejs/plugin-react": "^4.0",
    "tailwindcss": "^3.4",
    "autoprefixer": "^10.0",
    "postcss": "^8.0"
  }
}
```

MÓDULO IA — CHATBOT RAG (Detalle Técnico)

Arquitectura RAG Local

```
Documentos del sistema (Manuales, ayudas operativas)
  ↓
Chunking + Generación de Embeddings (text-embedding-ada / nomic-embed-text local)
  ↓
Almacenamiento en Vector Store (ChromaDB local / pgvector en Supabase en prod)
  ↓
Query del usuario → Embedding de la pregunta
  ↓
Búsqueda por similitud coseno → Top K chunks relevantes
  ↓
Prompt: "Basándote SOLO en los siguientes manuales del sistema FerreSync: {chunks}
        Responde esta pregunta operativa: {pregunta}"
  ↓
LLM (Ollama/llama3 local | OpenAI gpt-4o-mini en prod)
```



Respuesta precisa y sin alucinaciones

Documentos base de conocimiento del chatbot

- Manual de usuario del sistema (a crear durante el desarrollo)
 - Guía de procesos: cómo registrar venta, abono, devolución
 - FAQ operativo de la ferretería
-



CHECKLIST DE DESARROLLO (Orden sugerido)

Sprint 1 — Base del sistema

- ☐ Setup proyecto (estructura de carpetas, Docker, Git)
- ☐ Schema Prisma completo + primera migration
- ☐ Seed de datos iniciales (admin, productos de prueba)
- ☐ Auth: POST login, middleware JWT, middleware roles
- ☐ Layout frontend: sidebar, topbar, rutas protegidas
- ☐ Página de Login

Sprint 2 — Módulo de Inventario

- ☐ CRUD de productos (backend + frontend)
- ☐ Upload de facturas de compra (Multer local)
- ☐ Catálogo con filtros y paginación
- ☐ Alertas visuales de stock crítico en dashboard

Sprint 3 — Punto de Venta (POS)

- ☐ Buscador de productos en tiempo real
- ☐ Carrito con cálculo automático
- ☐ Registro de venta + descuento de stock (transacción atómica)
- ☐ Generación de ticket (PDF con Puppeteer o impresión directa)
- ☐ Historial de ventas con filtros

Sprint 4 — Cuentas por Cobrar

- ☐ CRUD de clientes
- ☐ Ventas a crédito desde el POS
- ☐ Estado de cuenta por cliente
- ☐ Registro de abonos con comprobante
- ☐ Resaltado visual de cuentas en mora

Sprint 5 — Operaciones Avanzadas

- ☐ Cierre de caja (arqueo diario + PDF)
- ☐ Devoluciones de mercancía
- ☐ Cronograma logístico (calendario)
- ☐ Gestión de personal y roles

Sprint 6 — IA, Reportes y Pulido

- ☐ Chatbot RAG (Ollama local)
- ☐ Reportes de ventas e inventario (PDF)
- ☐ Diseño final responsivo
- ☐ Testing de seguridad (rutas protegidas por rol)
- ☐ Documentación de la API (Swagger/Postman)

Sprint 7 — Migración a Producción

- ☐ Cambiar DATABASE_URL a Supabase
- ☐ Migrar uploads a Supabase Storage
- ☐ Cambiar IA a OpenAI
- ☐ Deploy backend en VPS Contabo (PM2 + Nginx)
- ☐ Deploy frontend en Vercel o VPS

PROMPT PARA INICIAR EL PROYECTO CON IA

Usa este prompt al inicio de cada sesión de desarrollo con un asistente IA:

Eres un desarrollador full-stack senior. Estamos construyendo "FerreSync", un sistema POS y gestión de inventario para una ferretería tradicional.

STACK TÉCNICO:

- Frontend: React 18 + Vite + TailwindCSS + React Router v6 + Axios + TanStack Query
- Backend: Node.js + Express.js + Prisma ORM
- BD: PostgreSQL 15 (local con Docker, migrar a Supabase en producción)
- Auth: JWT (jsonwebtoken) + bcrypt
- Archivos: Multer (local) → Supabase Storage (producción)
- IA: LangChain.js + Ollama local → OpenAI en producción

REGLAS DE ARQUITECTURA:

- API REST desacoplada: frontend consume backend via HTTPS/JSON
- Separar routes → controllers → services → prisma en el backend
- Autenticación JWT en todas las rutas protegidas
- Dos roles: ADMINISTRADOR y CAJERO (con permisos distintos)
- Transacciones atómicas en operaciones críticas (registrar venta = descontar stock + crear venta en una sola transacción)
- Variables de entorno en .env, nunca hardcodeadas

MÓDULOS: POS (ventas), Inventario, Cuentas por Cobrar (créditos/abonos), Cierre de Caja, Devoluciones, Cronograma Logístico, Gestión de Personal, Chatbot IA RAG.

CONTEXTO DE USUARIOS: Propietario de ferretería (admin) y cajeros con nivel básico de alfabetización digital. La UI debe ser simple, directa, con botones grandes y flujos de máximo 3 clics para una venta.

Ahora construyamos [DESCRIBE AQUÍ LO QUE NECESITAS]: ...

 HOJA DE RUTA DE MIGRACIÓN A PRODUCCIÓN

DESARROLLO LOCAL

PostgreSQL (Docker)

→

Multer + /uploads

→

Ollama + llama3

→

localhost:3001

→

localhost:5173

→

.env local

→

PRODUCCIÓN

Supabase PostgreSQL

Supabase Storage SDK

OpenAI gpt-4o-mini + pgvector

VPS Contabo (PM2 + Nginx + SSL)

Vercel / subdominio en VPS

Variables de entorno del servidor

Cambios requeridos en el código para migración:

1. `DATABASE_URL` en `.env` → string de conexión Supabase
 2. `uploadService.js`: cambiar de guardar en `/uploads` a `supabase.storage.from('bucket').upload(...)`
 3. `iaService.js`: cambiar `OllamaEmbeddings` por `OpenAIEmbeddings` + `vectorstore` `SupabaseVectorStore`
 4. Configurar PM2 en VPS: `pm2 start src/app.js --name ferresync-api`
 5. Configurar Nginx como proxy inverso al puerto 3001
-

Documento generado para: FerreSync / Cristian David Cárdenas García / UTS 2026